UNIDAD 3: VERIFICACIÓN Y VALIDACIÓN (V&V) CON HERRAMIENTAS

BLOQUE 1: FUNDAMENTOS DE CALIDAD Y GOBERNANZA (SEMANA 1)

1. ARQUITECTURA DE LOS PROCESOS DE CONTROL: VERIFICACIÓN Y VALIDACIÓN (V&V)

En el paradigma de la Ingeniería de Software moderna, el **Aseguramiento de la Calidad del Software (SQA - Software Quality Assurance)** no se concibe como una fase tardía o un filtro manual de cierre, sino como un sistema de gobernanza continua que atraviesa transversalmente toda la construcción del producto. Los pilares operativos de este sistema son los procesos de **Verificación y Validación (V&V)**, los cuales actúan como mecanismos de control de ingeniería independientes, con objetivos y métodos diferenciados pero acoplados.

Su distinción técnica responde a la dualidad formal planteada en la teoría de Barry Boehm:

- **Verificación:** ¿Estamos construyendo el software correctamente?
- **Validación:** ¿Estamos construyendo el software correcto?

1.1 Verificación: Control de Conformidad Estructural y Sintáctica

La verificación es un proceso de auditoría interna y de naturaleza matemática y lógica. Su propósito es certificar de manera estática o mediante pruebas de aislamiento que cada artefacto de software intermedio (módulos, esquemas, código fuente) se condice de forma idéntica con las especificaciones técnicas y los requerimientos de diseño impuestos en la fase previa de ingeniería.

- **Enfoque de Caja Blanca:** Analiza la estructura interna del código, el flujo de control, el acoplamiento entre objetos, la adherencia a patrones de diseño y el comportamiento sintáctico.
- **Mecanismos:** Se ejecuta mediante la combinación de análisis estático (inspección de planos sin ejecución del programa) y análisis dinámico atómico (pruebas de componentes aislados).
- **Meta técnica:** Garantizar la consistencia interna del sistema, la robustez de la arquitectura y la ausencia de desvíos lógicos respecto al diseño formal.

1.2 Validación: Control de Idoneidad Funcional y de Negocio

La validación es un proceso de auditoría externa y de naturaleza puramente operativa. Su propósito es certificar si el producto de software consolidado (como sistema integrado) resuelve efectivamente el problema de negocio o la necesidad real del usuario final dentro de su entorno operativo.

- **Enfoque de Caja Negra:** Prescinde del análisis de la estructura del código o la sintaxis; evalúa el comportamiento del software frente a estímulos operativos reales.
- **Mecanismos:** Se ejecuta exclusivamente mediante análisis dinámico, poniendo en marcha el sistema y confrontándolo con flujos de uso de extremo a extremo e inyecciones de datos que simulan la carga real de trabajo.
- **Meta técnica:** Asegurar el valor de negocio, la usabilidad operativa y que el sistema sea apto para el propósito para el cual fue financiado.

1.3 Matriz Comparativa de Ingeniería de V&V

Dimensión Técnica	**Verificación**	**Validación**
Pregunta Core	¿Estamos construyendo el producto correctamente?	¿Estamos construyendo el producto correcto?
Foco de Control	Cumplimiento estricto de especificaciones técnicas, planos de diseño y estándares lógicos.	Aptitud del sistema, satisfacción del cliente y resolución del problema real de negocio.
Naturaleza del Análisis	**Estático y Dinámico:** Inspecciones de código, walkthroughs y suites de	**Dinámico:** Ejecución del sistema consolidado frente a escenarios de negocio de extremo a extremo.

	pruebas de componentes individuales.	
Actores del Proceso	Desarrolladores, Líderes Técnicos, Arquitectos de Software y QA Automation Engineers.	Product Owners, Analistas de Negocio, Clientes y Usuarios Finales.
Artefactos Bajo Examen	Código fuente, esquemas de bases de datos, documentación de contratos de APIs (Swagger), diagramas UML.	Software operativo integrado corriendo en entornos locales de pruebas, réplicas o forks de desarrollo.

2. DEFINITION OF DONE (DOD) INDUSTRIAL DE LABORATORIO

La **Definition of Done (DoD)** o *Definición de Terminado* es un acuerdo de ingeniería explícito, formal, público y vinculante establecido por el equipo de desarrollo. Opera como un contrato de calidad de grado industrial y funciona como una compuerta estricta que prohíbe que cualquier incremento de código sea integrado a la rama principal de un repositorio si no cumple con la totalidad de los criterios técnicos definidos.

2.1 Desglose Técnico de los Criterios Obligatorios de una DoD de Laboratorio

En un entorno de desarrollo profesional, para que una tarea o funcionalidad se declare formalmente "Done" (Terminada), debe superar de manera automatizada o verificable los siguientes puntos de control locales:

1. **Integridad Absoluta de Compilación:** El nuevo código fuente o la modificación realizada debe compilar de forma limpia en el entorno local del desarrollador. No se tolera ningún error de compilación ni advertencias (*warnings*) críticas que apunten a futuras fallas de tipado o memoria.

2. **Validación de Pruebas Automatizadas:** El 100% de la suite de pruebas unitarias y funcionales locales asociadas al componente debe ejecutarse con resultado exitoso (en verde). La presencia de un solo test fallido veta automáticamente la integración.
3. **Umbrales Mínimos de Cobertura de Código:** El incremento de código debe cumplir con la métrica de cobertura estipulada para el proyecto (por ejemplo, certificar mediante reportes locales que al menos el 80% de las nuevas líneas de código han sido cubiertas por pruebas automáticas).
4. **Revisión por Pares Obligatoria (Code Review):** El código debe ser sometido a una inspección por parte de otro miembro técnico del equipo. Esta revisión audita que la solución no incorpore malas prácticas, respete los patrones arquitectónicos adoptados y no incremente innecesariamente la complejidad del sistema.
5. **Gobernanza Estricta de Estilo:** El código fuente debe pasar la validación sintáctica y visual de las reglas preconfiguradas del proyecto, garantizando que el diseño del texto sea uniforme e indistinguible entre desarrolladores.
6. **Sustento de la Documentación Técnica:** Cualquier cambio que altere contratos de comunicación debe registrarse de manera mandatoria. Esto incluye documentar funciones de alta complejidad, actualizar esquemas de bases de datos locales y regenerar las especificaciones de interfaces de APIs (como contratos Swagger u OpenAPI).

2.2 Desacoplamiento Conceptual: DoD vs. Criterios de Aceptación

- **La Definition of Done (DoD) es global y técnica:** Es invariable a lo largo del sprint o ciclo de desarrollo. Afecta a cada porción de software por igual, enfocándose en la calidad estructural del plano de ingeniería (ej. "el código debe estar libre de warnings y documentado").
- **Los Criterios de Aceptación son específicos y funcionales:** Se diseñan a medida para cada historia de usuario o requerimiento funcional específico. Evalúan el comportamiento lógico del negocio y varían en cada tarea (ej. "el campo numérico de transferencia debe rechazar valores negativos y procesar transacciones en menos de 2 segundos").

3. INGENIERÍA DE EVIDENCIAS Y GOBERNANZA DE REPOSITORIOS LOCALES

En la cultura de la ingeniería de software de alta confiabilidad, las afirmaciones subjetivas del tipo "el código ya funciona" o "el módulo está verificado" carecen de

validez técnica si no están respaldadas por **evidencias objetivas auditables**. Una evidencia es un artefacto digital inmutable, fechado y estructurado que demuestra científicamente la ejecución y el resultado exitoso de un control de calidad.

3.1 Tipología Completa de Evidencias Técnicas

- **Reportes de ejecución de pruebas:** Documentos o tableros analíticos que listan de manera exhaustiva la totalidad de los casos de prueba ejecutados. Deben detallar el identificador del test, la fecha, el entorno de ejecución local, el operador, el resultado obtenido (pass/fail) y la latencia o tiempo de respuesta en milisegundos.
- **Registros logs de pruebas automatizadas:** Archivos estructurados de salida (generalmente en formatos estándares XML, JSON o texto plano de consola) emitidos de forma directa por los frameworks de testing al finalizar la corrida de la suite de código.
- **Métricas analíticas de cobertura (Code Coverage):** Informes detallados generados por herramientas de cobertura que mapean el código fuente línea por línea, calculando matemáticamente la proporción de sentencias, bloques lógicos y ramas condicionales que fueron efectivamente recorridas por las pruebas automáticas.
- **Reportes de análisis estático estructural:** Diagnósticos automáticos que certifican que el código fuente fue escaneado en frío y se encuentra libre de vulnerabilidades de seguridad, deudas técnicas severas, complejidades ciclomáticas desmedidas y duplicación de líneas de código.

3.2 Trazabilidad de Cambios mediante Plantillas y Gobernanza de Repositorio

Como primera línea de defensa en el laboratorio de desarrollo, y para evitar el desorden operativo en los entornos locales o forks de los programadores, la gobernanza del repositorio exige la implementación estricta de dos mecanismos de control dentro de la plataforma de control de versiones:

A) Issue Templates (Plantillas Estructuradas de Incidencias)

Son archivos de configuración obligatorios (generalmente en formato Markdown) que estandarizan el ingreso de cualquier solicitud de cambio, bug report o requerimiento técnico en el repositorio. El uso de plantillas anula la ambigüedad y la carga de datos incompleta al forzar al emisor a completar secciones técnicas predefinidas.

Ejemplo formal de estructura para una plantilla de reporte de defectos:

- **Título:** Prefijo claro indicando el tipo de problema (ej. BUG: [Breve resumen del fallo]).
- **Descripción del Problema:** Resumen técnico detallado del comportamiento anómalo detectado en el sistema.
- **Pasos Detallados para Reproducir:** Lista ordenada y numerada de las acciones exactas que disparan la falla de forma determinista.
- **Comportamiento Esperado:** Qué debió haber hecho el sistema según lo establecido en las especificaciones técnicas.
- **Comportamiento Observado:** Qué hizo realmente el sistema (error de consola, congelamiento de interfaz, desborde de memoria, etc.).
- **Evidencias Adjuntas:** Sección destinada a la inclusión de capturas de pantalla, archivos de logs recuperados localmente o reportes de consola.

B) Labels (Etiquetas Sistémicas de Categorización)

Las etiquetas son metadatos de clasificación que se asignan formalmente a las incidencias (*issues*) y propuestas de cambios (*pull requests*) dentro del repositorio. Su función primordial es establecer una trazabilidad visual y operativa clara que permita organizar el esfuerzo técnico y auditar el cumplimiento de la Definition of Done antes de mezclar código.

Las categorías mandatorias de etiquetas para la gobernanza del laboratorio incluyen:

- bug: Asignada de forma exclusiva a fallos lógicos o funcionales confirmados que rompen un comportamiento esperado.
- technical-debt: Aplicada a refactorizaciones, código de alta complejidad o desvíos de estilo identificados por linters que deben sanearse.
- documentation: Destinada a tareas enfocadas en la actualización de contratos de APIs, diagramas de arquitectura o especificaciones Swagger.
- feature: Utilizada para el desarrollo e implementación de nuevas capacidades lógicas del negocio.